

Evaluación de EDA II 2026

Sistema de evaluación

Este documento contiene 20 ejercicios de dificultad variable, agrupados por temas de la asignatura. Cada ejercicio tiene una puntuación asignada; la nota final se determina según el puntaje total acumulado:

Nota	Puntaje mínimo
3	15 puntos
4	20 puntos
5	25 puntos

Para alcanzar cualquier nota, es condición necesaria haber resuelto **al menos un ejercicio de cada tema**. El incumplimiento de esta condición implica nota 2, independientemente del puntaje acumulado.

Salvo los ejercicios 16, 17 y 18 —que consisten en demostraciones puramente teóricas— en todos los demás se debe:

- justificar la **correctitud** del algoritmo propuesto,
- presentar un **pseudocódigo** del mismo, y
- analizar la **complejidad temporal** en el peor caso.

Tabla de ejercicios

Ej.	Nombre	Puntos
<i>BFS y DFS</i>		
1	Completando componentes	2
2	El bosque de Alberto	3
3	El alfil bloqueado	5
<i>Puntos de Articulación y Aristas Puente</i>		
4	Pares desconectados	2
5	Orientación fuertemente conexa	4
6	Contando puentes en caminos	4
<i>Árbol Abarcador de Costo Mínimo</i>		
7	AACM multiplicativo	2
8	Arista imprescindible	3
9	Servidores en grupos	3
<i>DAG, Orden Topológico y Componentes Fuertemente Conexas</i>		
10	Bailes en círculo	3
11	Pociones	4
12	2-SAT	5
<i>Caminos de Costo Mínimo</i>		
13	El edicto del Rey Alberto	2
14	Caminos con peajes internos	3
15	El servidor central	3
<i>Flujo Máximo – Corte Mínimo</i>		
16	Flujo máximo sin aumentativos	1
17	Del residual al corte	1
18	Corte saturado implica máximo	1
19	Escape por la frontera	4
20	La red social	5
Total disponible		60

Ejercicio 1 – Completando componentes. Dado un grafo $G = \langle V, E \rangle$ no necesariamente conexo, diseñe un algoritmo que permita determinar la cantidad de aristas que son necesarias insertar en G para que cada una de las componentes conexas sea un subgrafo completo. La complejidad temporal de su algoritmo debe ser $\mathcal{O}(|V| + |E|)$.

Ejercicio 2 – El bosque de Alberto. El mapa de un bosque se puede representar como una matriz de N filas y M columnas; cada casilla puede estar vacía ($\cdot$) o contener un árbol ($\#$). Todos los días Alberto camina desde la posición $(1, 1)$ hasta la posición (N, M) ; en un paso puede moverse desde su posición actual en una de las cuatro direcciones: arriba, abajo, derecha o izquierda.

Como parte de un plan para la protección de los bosques, Alberto planea plantar árboles en algunas de las casillas vacías, pero quiere hacerlo de tal forma que siga existiendo una forma de caminar desde $(1, 1)$ hasta (N, M) .

Diseñe un algoritmo que permita determinar la **máxima cantidad** de árboles que Alberto puede plantar. La complejidad temporal de su algoritmo debe ser $\mathcal{O}(NM)$.

Ejercicio 3 – El alfil bloqueado. Dado un tablero de ajedrez de N filas y N columnas, cada casilla puede estar vacía ($\cdot$), contener un alfil blanco (A) o contener un peón blanco (P). Se garantiza que el tablero contiene exactamente un alfil y no se permiten mover o retirar los peones. Se necesita mover el alfil desde su casilla actual hasta la casilla (X, Y) siguiendo las reglas de ajedrez.

Diseñe un algoritmo que permita determinar la **mínima cantidad de movimientos** necesarios para llevar el alfil a la casilla (X, Y) . La complejidad temporal debe ser $\mathcal{O}(N^2)$.

Nota: Un alfil en la casilla (i, j) puede ir a la casilla $(i \pm d, j \pm d)$ en un movimiento, para cualquier entero positivo d , siempre que:

- la casilla $(i \pm d, j \pm d)$ exista en el tablero, y
- para todo entero positivo $l \leq d$, la casilla intermedia $(i \pm l, j \pm l)$ esté vacía.

Puntos de Articulación y Aristas Puente

Ejercicio 1 – Pares desconectados. Sea $G = (V, E)$ un grafo no dirigido y conexo. Se desea calcular, para cada arista $e \in E$, el número de pares de vértices distintos que se desconectan cuando se elimina dicha arista del grafo. Diseñe un algoritmo que resuelva el problema en tiempo $\mathcal{O}(|V| + |E|)$. Demuestre la correctitud de su algoritmo y justifique su complejidad temporal.

Ejercicio 2 – Orientación fuertemente conexa. Sea $G = (V, E)$ un grafo no dirigido. Diseñe un algoritmo que determine si es posible orientar las aristas de G de forma tal que, para todo par de vértices $i, j \in V$, existan caminos dirigidos en ambas direcciones, es decir, $i \rightsquigarrow j$ y $j \rightsquigarrow i$. Si la respuesta es afirmativa, el algoritmo debe devolver una asignación válida de direcciones para las aristas. Demuestre la correctitud del algoritmo y justifique que su complejidad temporal es $\mathcal{O}(|V| + |E|)$.

Ejercicio 3 – Contando puentes en caminos. Se desea implementar una estructura de datos que, dado un grafo no dirigido y conexo $G = (V, E)$, permita realizar consultas de la forma $\text{CANTPUENTES}(G, a, b)$, donde $a, b \in V$, para determinar la cantidad de aristas puente que hay en todo camino simple de a a b .

- a) Demuestre que todos los caminos simples entre a y b tienen la misma cantidad de aristas puente.
- b) Diseñe una estructura de datos que permita responder consultas $\text{CANTPUENTES}(G, a, b)$ en tiempo $\mathcal{O}(P)$, donde P es la cantidad de aristas puente de G . La complejidad temporal de inicializar la estructura debe ser $\mathcal{O}(|V| + |E|)$.

Explique la construcción de la estructura, demuestre su correctitud y justifique las complejidades temporales requeridas.

Árbol Abarcador de Costo Mínimo

Ejercicio 1 – AACM multiplicativo. Sea $w: E \rightarrow \mathbb{R}$ la función de costo de las aristas de G , con $w(e) > 1$ para toda $e \in E$. Suponga que se define el **costo de un árbol** abarcador T como el *producto* de los costos de sus aristas:

$$W(T) = \prod_{e \in T} w(e)$$

en lugar de la suma habitual. Implemente un algoritmo que, dado un grafo $G = \langle V, E \rangle$ con la función de costo w definida anteriormente, permita determinar el Árbol Abarcador de Costo Mínimo (AACM) de G de forma eficiente. La complejidad temporal de su algoritmo debe ser $\mathcal{O}(|E| \log |V|)$.

Ejercicio 2 – Arista imprescindible. Sea $G = \langle V, E \rangle$ un grafo no dirigido, ponderado y conexo. Diseñe un algoritmo eficiente que permita determinar si una arista $e \in E$ dada **aparece necesariamente en todo** AACM de G . La complejidad temporal de su algoritmo debe ser $\mathcal{O}(|E| \log |V|)$.

Ejercicio 3 – Servidores en grupos. Tienes n servidores conectados entre sí; cada conexión tiene un peso. Se deben dividir en exactamente k grupos, donde el **costo de un grupo** es la suma de los pesos de las conexiones dentro de ese grupo. Diseñe un algoritmo eficiente que minimice la **suma total de costos** de los k grupos. La complejidad temporal de su algoritmo debe ser $\mathcal{O}(|E| \log |V|)$.

DAG, Orden Topológico y Componentes Fuertemente Conexas

Ejercicio 1 – Bailes en círculo. A un festival asistieron n personas y decidieron realizar varios bailes en círculos. Cada baile está formado por al menos 2 personas, y cada persona tiene exactamente dos vecinos en el círculo. En el caso de un círculo de solo 2 personas, cada una tiene al otro como único vecino (dos veces). Cada participante recuerda exactamente a una persona que era su vecina en el baile en el que participó. A partir de esta información, diseñe un algoritmo que determine cuál es el **mínimo número posible de bailes en círculos** que pudieron haberse formado. El algoritmo debe tener complejidad temporal $\mathcal{O}(n)$.

Ejercicio 2 – Pociones. A Alberto le gusta mezclar pociones. Existen n tipos de pociones. Dada una lista $c_1, c_2, \dots, c_n$, donde c_i representa el costo en monedas de comprar directamente una poción de tipo i , se desea determinar el menor costo posible para obtener cada tipo de poción.

Algunos tipos de pociones pueden obtenerse mezclando varias otras pociones; las pociones utilizadas en una mezcla se consumen. Cada tipo de poción tiene a lo sumo una receta asociada, y se garantiza que ninguna poción puede obtenerse a partir de sí misma mediante una o más mezclas.

Además, se proporciona un arreglo P que lista los tipos de pociones que Alberto ya posee en cantidad ilimitada; obtener una poción cuyo tipo aparece en P tiene costo 0.

Diseñe un algoritmo que, dado el conjunto P , la lista de costos c_i y las recetas disponibles, determine para cada tipo de poción i el **mínimo número de monedas** que Alberto debe gastar para obtener una poción de tipo i . La complejidad temporal debe ser $\mathcal{O}(n + M)$, donde M es la suma de las cantidades de ingredientes en todas las recetas.

Ejercicio 3 – 2-SAT. Dada una fórmula en FNC¹ donde el número de literales en cada disyunción es menor o igual a 2, determinar si es satisfacible o no se conoce como el problema **2-SAT**. Se recibe una fórmula de la forma:

$$F(x_i) = (x_0 \vee \neg x_1) \wedge (x_2 \vee x_1) \wedge (\neg x_3 \vee x_4) \wedge \dots$$

donde $X = \{x_i\}$ es el conjunto de variables y C el conjunto de cláusulas.

Una fórmula es *satisfacible* si existe una asignación booleana sobre las variables x_i que haga verdadera $F(x_i)$; en caso contrario se dice *insatisfacible*.

Diseñe un algoritmo que resuelva 2-SAT en $\mathcal{O}(|X| + |C|)$ y que, cuando F sea satisfacible, brinde una asignación de las variables x_i que haga la fórmula verdadera.

Caminos de Costo Mínimo

Ejercicio 1 – Caminos con peajes internos. Sea $G = (V, E)$ un grafo dirigido donde cada arista $e \in E$ posee un costo de tránsito estrictamente positivo $w(e) > 0$. Adicionalmente, cada vértice $v \in V$ tiene asociado un costo de activación interno $c(v) \in \mathbb{R}$, el cual puede ser positivo o negativo (donde un valor negativo representa un reembolso o descuento neto al transitar por dicho nodo). El costo total de un camino se define como la suma de los costos de sus aristas más los costos de activación de los nodos intermedios visitados. Diseñe un algoritmo que determine el **costo mínimo** para viajar desde un vértice origen s hasta un vértice destino t . La complejidad temporal de su algoritmo debe ser $\mathcal{O}(|V| \cdot |E|)$.

Ejercicio 2 – El edicto del Rey Alberto. En el reino de MatCom, el Rey Alberto — máximo guardián de la complejidad computacional — ha decretado que todo problema del reino debe ser resuelto en tiempo óptimo. Desde su trono en la capital, Alberto envía mensajeros a cada aldea del reino a través de una red de **caminos dirigidos**, cada uno con un costo medido en **operaciones elementales**.

Las aldeas del reino están agrupadas en tres clases de complejidad según cuántas operaciones pueden tolerar antes de declarar que un problema es *intratable*:

¹Una conjunción de cláusulas, donde cada cláusula es una disyunción de literales.

- **Polinómicas**, que toleran hasta C_P operaciones.
- **Logarítmicas**, las más exigentes, que toleran hasta C_L operaciones.
- **Exponenciales**, las más permisivas, que toleran hasta C_E operaciones.

Si el costo mínimo para que el mensajero del Rey Alberto llegue a una aldea supera el umbral de su clase, esa aldea declara el problema **NP-difícil** y eleva una queja formal al trono.

Diseña un algoritmo $\mathcal{O}(|E| \log |V|)$ que determine cuántas aldeas del reino elevan una queja formal al Rey Alberto.

Ejercicio 3 – El servidor central. Una compañía de telecomunicaciones desea instalar un servidor central en una red de computadoras interconectadas, modelada como un grafo dirigido ponderado $G = (V, E)$ con costos de latencia positivos en las aristas $w(e) > 0$. Para garantizar la calidad del servicio, se busca seleccionar un vértice óptimo $v^* \in V$ que minimice la máxima latencia de comunicación hacia cualquier otro nodo de la red. Es decir, si $d(u, v)$ representa la distancia mínima entre dos nodos cualesquiera, se desea hallar un v^* que minimice la función $\max_{u \in V} d(v^*, u)$. Diseña un algoritmo que determine la **ubicación exacta del servidor central**. La complejidad temporal de su algoritmo debe ser $\mathcal{O}(|V|^3)$.

Flujo Máximo - Corte Mínimo

Ejercicio 1 – Flujo máximo sin aumentativos. Sea $G = (V, E)$ una red de flujo con una fuente $s \in V$ y un receptor $t \in V$, y sea f un flujo admisible en G . Demuestre que si f es un flujo máximo en G , entonces no existen caminos aumentativos desde s hasta t en la red residual G_f .

Ejercicio 2 – Del residual al corte. Sea $G = (V, E)$ una red de flujo con fuente $s \in V$ y receptor $t \in V$, y sea f un flujo admisible en G . Suponga que en la red residual G_f no existen caminos aumentativos desde s hasta t . Defina un corte (S, T) y demuestre que el valor del flujo cumple exactamente la condición $|f| = c(S, T)$.

Ejercicio 3 – Corte saturado implica máximo. Sea $G = (V, E)$ una red de flujo con fuente $s \in V$ y receptor $t \in V$, y sea f un flujo admisible en G . Suponga que existe un corte (S, T) tal que $|f| = c(S, T)$. Demuestre que f es un flujo máximo en G .

Ejercicio 4 – Escape por la frontera. Dada una matriz de $n \times n$ interpretada como un grafo no dirigido (donde cada vértice interno tiene exactamente 4 vecinos ortogonales) y dados m puntos de inicio $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$, diseñe un algoritmo que determine si existen m caminos disjuntos en vértices que comiencen en dichos puntos de inicio y terminen en el borde de la matriz. La complejidad temporal de su algoritmo debe ser $\mathcal{O}(|E| \cdot m)$, donde E es el conjunto de aristas de la matriz.

Ejercicio 5 – La red social. En una red social con un conjunto de usuarios U , se registran dos números enteros para cada usuario que representan su cantidad exacta de seguidos (grado de salida) y su cantidad de seguidores (grado de entrada). Diseña un algoritmo que permita determinar si existe un grafo dirigido sin lazos ni aristas paralelas que cumpla exactamente con dicha descripción. La complejidad temporal de su algoritmo debe ser $\mathcal{O}(|U|^4)$.